

Working with Linux

This is the second part of the tutorial/workshop on the Linux operating system. It is assumed that you have completed, and understand, the concepts and commands in the first part, “An Introduction to Linux”. Most of the information presented in this tutorial will build on that.

Shell “Tab Completion”

In the first tutorial we learned that the Linux “shell” is the command interpreter which reads your input and executes commands. We only used the shell in the simplest way, by typing the full name of the command to be executed, and likewise completely typing out any command-line flags, options or arguments. But the shell contains a feature, called “tab completion”, which can save you a lot of typing, and can also help you find out what commands or files are available for your use.

Command Completion

When you are typing a command at the shell prompt, you can hit <Tab> at any time to get the shell to show you all available commands which match what you have entered up to that point. For example, if you are going to edit a text file, you could type “nan” and then press <Tab>. In this case, because there is only one Linux command which starts with the letters “nan”, the shell will fill in the rest of the command name, and move to a point one space past the end of the command name so that you can enter the name of the file you want to edit, like this:

```
pi@raspberrypi:~ $ nan
```

```
pi@raspberrypi:~ $ nano
```

If there is more than one Linux command which matches your input, the shell will not do anything the first time you press <Tab>, but if you press it twice the shell will then display a list of all commands which match your input so far, so that you know what is possible, and will then just wait for the rest of your input. (If the number of matching commands is very large, the shell will ask of you want to see the entire list)

```
pi@raspberrypi:~ $ ls
ls          lsb_release  lsipc       lsmem       lsof
lsattr      lscpu        lslocks     lsmod       lspgpot
lsblk       lsintramfs   lslogins    lsns        lsusb
pi@raspberrypi:~ $ ls
```

You might be surprised to see that there are so many Linux commands which start with “ls”, but this is actually a common prefix for commands which produce a “list” of something. We have only seen the common “directory listing” command until now, but if you are interested in finding out what some of these other commands do, you can always look them up with the “man” command. For example, if you try “man lsof” you will find that this command gives a list of open files.

Filename Completion

Filename completion is another very useful form of tab completion in the Linux shell. In this case, when you have typed the name of a command and you are then typing the name of a file as an argument for that command, you can again enter any number of characters for the beginning of the filename and then hit <Tab> to get a list of matching files. If there is only one file which matches the letters you have entered, the name will be completed on the command line; if there is more than one matching file, the shell will display a list of the matches and then return to the point on the command input where you hit <Tab>.

For example, when loading the Raspbian operating system on a Raspberry Pi, you have to download an ISO image and then copy it to the micro-SD memory card. The name of that ISO file is rather long and clumsy, something like 2019-07-10-raspbian-buster-lite.zip. If you have to type this name very often, your fingers are likely to get very tired! Fortunately, shell tab completion can save you a lot of typing. Suppose you just wanted a “long” directory listing of this file, so you could see what size it is. You could enter just the command name and the first few characters of the filename:

```
pi@raspberrypi:~/Downloads $ ls -l 2019
```

Then hit <Tab> to get the shell to fill in the rest of the name for you:

```
pi@raspberrypi:~/Downloads $ ls -l 2019-07-10-raspbian-buster.zip
```

If there is more than one file which matches the characters you have typed when you hit <Tab>, the shell will fill in all of the leading characters which are common to the files and return to the input at the end of the characters it has filled in:

```
pi@raspberrypi:~/Downloads $ ls -l 2019-07-10-raspbian-buster
```

If you are not sure exactly what filename you need at this point, you can simply hit <Tab> twice, and the shell will present you with a list of matching filenames and then return to command input at the same point.

```
pi@raspberrypi:~/Downloads $ ls -l 2019-07-10-raspbian-buster
2019-07-10-raspbian-buster-full.zip 2019-07-10-raspbian-buster.zip
2019-07-10-raspbian-buster-lite.zip
pi@raspberrypi:~/Downloads $ ls -l 2019-07-10-raspbian-buster
```

You could then continue entering enough letters to uniquely identify the file you wanted.

Shell Input and Output: Redirect and Pipe

Redirecting Output

As you have seen so far, the Linux shell (command interpreter) normally reads its input from the keyboard, and writes its output to the console or terminal emulator. However, this is only a "default" operation, the actual concept of process input and output in Linux is defined as abstract objects, known as "standard input" (or stdin) and "standard output" (stdout), which are connected to your keyboard and display when you login. It is possible to "redirect" the input and/or output for any command, using special symbols on the command line.

You can use the ">" symbol on the command line to tell the shell to send the output of a command to a file, rather than the console or terminal. This is shown in the following sequence, where the "ls" command is first run without redirect, so the directory contents are listed on the console, then it is run again with the output redirected to a file called "files.list".

```
pi@raspberrypi:~ $ ls
Desktop      DirTwo      Downloads  Music      NewDirectory  Public      Videos
DirectoryOne Documents  MagPi      MyProject  Pictures      Templates
pi@raspberrypi:~ $ ls > files.list
pi@raspberrypi:~ $ ls
Desktop      DirTwo      Downloads  MagPi  MyProject  Pictures  Templates
DirectoryOne Documents  files.list Music  NewDirectory  Public  Videos
pi@raspberrypi:~ $ ls -l files.list
-rw-r--r-- 1 pi pi 127 Jul 10 13:53 files.list
pi@raspberrypi:~ $ cat files.list
Desktop
DirectoryOne
DirTwo
Documents
Downloads
files.list
MagPi
Music
MyProject
NewDirectory
Pictures
Public
Templates
Videos
pi@raspberrypi:~ $
```

It is very important to understand that it is the **shell** which does this; the "ls" command simply writes its output to stdout, it has no idea that anything special is being done with that output. This is called "output redirection".

It is also important to know that this form of output redirection will always create a new file to contain the output, so if there is already a file with this name it will be destroyed and replaced by the new file containing the redirected output.

If you want to preserve the contents of an existing file, you can use the ">>" form of the output redirection command. This tells the shell to "append" the output to a file. If you followed the commands above with this:

```
pi@raspberrypi:~ $ ls -l >> files.list
pi@raspberrypi:~ $ ls -l files.list
-rw-r--r-- 1 pi pi 844 Jul 10 13:54 files.list
pi@raspberrypi:~ $
```

After these commands your file would contain a "short" directory list (only the names), followed by a "long" directory list.

If the file named after the ">>" redirection does not exist, it will simply be created and the new output will be written to it.

Redirecting Input

The input of the shell can also be redirected, in a very similar way to redirecting the output, using the "<" character. This is not as commonly used as output redirection, so we will not try to include an example here. But the concept of input redirection is actually very important to the next topic we are going to cover.

Pipes

All (proper) Linux commands are written to use these abstract objects for their input and output (stdin and stdout), rather than being hard-coded to use the keyboard and screen. We have just seen that the Linux shell is able to redirect either input or output. The next logical step is to redirect the output of one process to the input of another process, without having to use any kind of "temporary file" to hold the data that is being passed between the two processes. By doing this, it is possible to "chain" commands together, using a "pipe" (represented by the "|" character) between the commands. For example, consider the Linux "ps" (process status) command. When used with command line options to list all processes currently running on the system it will produce much more output than can be displayed on a typical terminal or console, like this:

```
11231 ?      R      0:01 [kworker/0:0-events]
11262 ?      I      0:00 [kworker/2:0-mm_percpu_wq]
11277 ?      I      0:00 [kworker/3:1-events_freezable]
11350 ?      I      0:00 [kworker/3:3-mm_percpu_wq]
11446 ?      I      0:00 [kworker/2:1-mm_percpu_wq]
11447 ?      I<    0:00 [kworker/2:2H-kblockd]
11478 ?      I<    0:00 [kworker/0:1H-mmc_complete]
11479 ?      I<    0:00 [kworker/1:1H-kblockd]
11480 ?      I      0:00 [kworker/0:1-events]
11494 ?      I<    0:00 [kworker/3:0H]
11588 ?      I      0:00 [kworker/3:0-mm_percpu_wq]
11612 ?      I      0:00 [kworker/2:2-mm_percpu_wq]
11639 ?      I      0:00 [kworker/0:3]
11641 ?      I<    0:00 [kworker/2:1H]
11642 ?      I<    0:00 [kworker/1:0H-kblockd]
11668 ?      I      0:00 [kworker/1:1-mm_percpu_wq]
11670 ?      I      0:00 [kworker/1:3-cgroup_destroy]
11677 ?      I<    0:00 [kworker/3:2H-kblockd]
11678 ?      I      0:00 [kworker/3:2-events]
11681 ?      S      0:00 [scsi_eh_0]
11682 ?      I<    0:00 [scsi_tmf_0]
11683 ?      S      0:00 [usb-storage]
11723 pts/0    R+     0:00 ps ax
pi@raspberrypi:~/Downloads $
```

You could get around this problem by redirecting the output of "ps" to a file, and then using the "more" utility to display that file:

```
pi@raspberrypi:~/Downloads $ ps ax > procs.txt
pi@raspberrypi:~/Downloads $ more procs.txt
```

```
PID TTY      STAT   TIME COMMAND
  1 ?        Ss      0:11 /lib/systemd/systemd --system --deserialize 18
  2 ?        S        0:00 [kthreadd]
  3 ?        I<       0:00 [rcu_gp]
  4 ?        I<       0:00 [rcu_par_gp]
  8 ?        I<       0:00 [mm_percpu_wq]
  9 ?        S        0:01 [ksoftirqd/0]
 10 ?        I        0:03 [rcu_sched]
 11 ?        I        0:00 [rcu_bh]
 12 ?        S        0:00 [migration/0]
 13 ?        S        0:00 [cpuhp/0]
 14 ?        S        0:00 [cpuhp/1]
 15 ?        S        0:00 [migration/1]
 16 ?        S        0:00 [ksoftirqd/1]
 19 ?        S        0:00 [cpuhp/2]
 20 ?        S        0:00 [migration/2]
 21 ?        S        0:00 [ksoftirqd/2]
 24 ?        S        0:00 [cpuhp/3]
 25 ?        S        0:00 [migration/3]
 26 ?        S        0:00 [ksoftirqd/3]
 29 ?        S        0:00 [kdevtmpfs]
 30 ?        I<       0:00 [netns]
 34 ?        S        0:00 [khungtaskd]
--More-- (8%)
```

However, this can be done much more easily by combining these two commands into a single command line, using a Linux "pipe" (pipeline):

```
pi@raspberrypi:~/Downloads $ ps ax | more
```

This command pipeline will produce exactly the same results as the previous two-step command sequence, but with only one command and without creating a temporary file (which you would have to remember to delete after you finished).

More Complex Command Chains

The pipe character "|" can be repeated on a command line as many times as necessary, and combined with redirection of standard input or output, because each instance simply redirects the standard output and standard input of the two processes on each side of it. This can lead to rather complex-looking command lines, such as the one shown below, but if you look at it carefully you can see that it is nothing more than a sequence of simple Linux commands with the output of each one redirected to the input of the next:

```
pi@raspberrypi:~ $ ps ax | sed -e 's/\s\s*/\t/g' | cut -f 6 | sort | uniq -c | sort -r | head
 3 /usr/sbin/apache2
 3 [mmal-vchiq]
 3 bash
 2 /usr/bin/ssh-agent
 2 /usr/bin/dbus-daemon
 2 sort
 2 [SMIO]
 2 avahi-daemon:
 1 [xprtiod]
 1 xcompmgr
pi@raspberrypi:~ $
```

Can you use the “man” command to find out what each step of the pipeline does, and thus figure out what the final output of this pipeline shown here actually is?

Processes on Linux

Linux is, of course, a multi-processing operating system. That means that there can (and will) be many different processes running on your machine at the same time. In fact, it is not unusual to have 150 or more processes running, even when your machine appears not to be doing anything! You can see what processes are running by using the “ps” (process status) command. If you type the command with no command line arguments, you will get a list of only processes which belong to you:

```
pi@raspberrypi:~ $ ps
  PID TTY          TIME CMD
  895 pts/0        00:00:00 bash
 1873 pts/0        00:00:00 ps
pi@raspberrypi:~ $
```

If you add the “-ef” options, you will get a list of all of the processes currently running, which is considerably longer, just as we saw above with the “ps ax” command, so once again you can pipe the output to “more” (or “less”) to have it presented one screen at a time:

```
UID          PID    PPID    C  STIME TTY          TIME CMD
root           1         0  0   09:59 ?        00:00:03 /sbin/init splash
root           2         0  0   09:59 ?        00:00:00 [kthreadd]
root           3         2  0   09:59 ?        00:00:00 [rcu_gp]
root           4         2  0   09:59 ?        00:00:00 [rcu_par_gp]
root           5         2  0   09:59 ?        00:00:00 [kworker/0:0-events]
root           8         2  0   09:59 ?        00:00:00 [mm_percpu_wq]
root           9         2  0   09:59 ?        00:00:00 [ksoftirqd/0]
root          10         2  0   09:59 ?        00:00:00 [rcu_sched]
root          11         2  0   09:59 ?        00:00:00 [rcu_bh]
root          12         2  0   09:59 ?        00:00:00 [migration/0]
root          13         2  0   09:59 ?        00:00:00 [cpuhp/0]
root          14         2  0   09:59 ?        00:00:00 [cpuhp/1]
root          15         2  0   09:59 ?        00:00:00 [migration/1]
root          16         2  0   09:59 ?        00:00:00 [ksoftirqd/1]
root          19         2  0   09:59 ?        00:00:00 [cpuhp/2]
root          20         2  0   09:59 ?        00:00:00 [migration/2]
root          21         2  0   09:59 ?        00:00:00 [ksoftirqd/2]
root          24         2  0   09:59 ?        00:00:00 [cpuhp/3]
root          25         2  0   09:59 ?        00:00:00 [migration/3]
root          26         2  0   09:59 ?        00:00:00 [ksoftirqd/3]
root          27         2  0   09:59 ?        00:00:00 [kworker/3:0-mm_percpu_wq]
root          29         2  0   09:59 ?        00:00:00 [kdevtmpfs]
--More--
```

Software Management

There are literally thousands of software packages containing utilities, applications, libraries and other optional software available for Linux systems. System administrators use the “apt” command to manage those packages – for querying which packages are installed, installing new packages and removing installed packages which are no longer needed.

For example, you might want to install the “apache” web server. The package name for that is actually “apache2” (to get the latest version), so the command would be:

```
pi@raspberrypi:~ $ sudo apt install apache2
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libapr1 libaprutil1
  libaprutil1-dbd-sqlite3 libaprutil1-ldap ssl-cert
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom
  openssl-blacklist
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libapr1 libaprutil1
  libaprutil1-dbd-sqlite3 libaprutil1-ldap ssl-cert
0 upgraded, 9 newly installed, 0 to remove and 0 not upgraded.
Need to get 1,990 kB of archives.
After this operation, 6,229 kB of additional disk space will be used.
Do you want to continue? [Y/n]
```

After you enter “Y” at this point (or simply press return, because Y is the default), the apt utility will download and install the packages; while that is happening you will see a series of status messages scrolling by on the console.

Linux Networking Commands

Networking is an important part of the use of Linux computers today. The following are a few of the Linux commands which provide information and management for network operation.

ping

One of the simplest, yet most versatile and useful of the networking commands, the “ping” utility is used to send an ICMP echo request to a network host. That host may be physically located anywhere, from the same room you are in to the opposite side of the world – in fact, it could also be your own computer! You give the IP address or hostname that you want to ping on the command line:

```
pi@raspberrypi:~ $ ping www.hva.nl
PING hvacms-prd-www.lb.uva.nl (145.18.12.47) 56(84) bytes of data.
64 bytes from hvacms-prd-www.lb.uva.nl (145.18.12.47): icmp_seq=1 ttl=56 time=23.5 ms
64 bytes from hvacms-prd-www.lb.uva.nl (145.18.12.47): icmp_seq=2 ttl=56 time=23.1 ms
64 bytes from hvacms-prd-www.lb.uva.nl (145.18.12.47): icmp_seq=3 ttl=56 time=23.4 ms
64 bytes from hvacms-prd-www.lb.uva.nl (145.18.12.47): icmp_seq=4 ttl=56 time=23.5 ms
^C
--- hvacms-prd-www.lb.uva.nl ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 8ms
rtt min/avg/max/mdev = 23.117/23.370/23.494/0.241 ms
pi@raspberrypi:~ $
```

```
pi@raspberrypi:~ $ ping 184.50.171.195
PING 184.50.171.195 (184.50.171.195) 56(84) bytes of data.
64 bytes from 184.50.171.195: icmp_seq=1 ttl=51 time=39.6 ms
64 bytes from 184.50.171.195: icmp_seq=2 ttl=51 time=136 ms
64 bytes from 184.50.171.195: icmp_seq=3 ttl=51 time=41.5 ms
64 bytes from 184.50.171.195: icmp_seq=4 ttl=51 time=40.3 ms
64 bytes from 184.50.171.195: icmp_seq=5 ttl=51 time=40.3 ms

--- 184.50.171.195 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 10ms
rtt min/avg/max/mdev = 39.618/59.538/135.972/38.222 ms
^Cpi@raspberrypi:~ $
```

The output of the ping command will tell you if it received a reply from the host, and if so it tells you how long it took to get that reply.

While this might seem like a very trivial utility at first, it actually provides you with some important information that can be very useful:

- It tells you that your computer has a working network connection
- It tells you that the computer you are pinging is running and connected to the internet
- It tells you that there is a network path between these two computers

This can be a very useful start when you are faced with network problems, because it will either eliminate or confirm some of the most obvious possibilities.

The “ping” command has a lot of useful options – try looking at “man ping” to see what they are. For example, what does the “-c” option do on the Linux ping command?

This also points out one of the unfortunate areas of differences between Linux and Windows utilities – the Windows version of ping uses different options (that means different command line letters) to implement some of the same functions as the Linux ping command. To see an example of this, find out what the Windows ping option equivalent to Linux “ping -c 5” would be.

IP Names and Addresses

As you have learned in your Infrastructure class, every device connected to the Internet must have an IP address. Most devices will also have a name, often called a “host name”. You can see a simple example of this in the “ping” command that you just tried. Although you entered “ping www.hva.nl”, the output of the ping command said that it was communicating with 145.18.12.47. There are Linux utilities to help you convert between IP addresses and Names, or the other way around.

Some Linux distributions do not include the DNS lookup utilities in the base installation, so you would have to add these to your machine. Fortunately, that is not difficult, and it takes very little time. All you need to do is run “sudo apt install dnsutils”.

IMPORTANT NOTE: The “apt” command requires root privileges, so you *must* use “sudo” to run it!

```
pi@raspberrypi:~ $ sudo apt install dnsutils
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libirs161
Suggested packages:
  rblcheck
The following NEW packages will be installed:
  dnsutils libirs161
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 596 kB of archives.
After this operation, 952 kB of additional disk space will be used.
Do you want to continue? [Y/n]
Get:1 http://debian.anexia.at/raspbian/raspbian buster/main armhf libirs161 armhf 1:9.11.5.P4+dfsg-5.1 [235 kB]
Get:2 http://debian.anexia.at/raspbian/raspbian buster/main armhf dnsutils armhf 1:9.11.5.P4+dfsg-5.1 [361 kB]
Fetched 596 kB in 1s (1,008 kB/s)
Selecting previously unselected package libirs161:armhf.
(Reading database ... 145174 files and directories currently installed.)
Preparing to unpack .../libirs161_1%3a9.11.5.P4+dfsg-5.1_armhf.deb ...
Unpacking libirs161:armhf (1:9.11.5.P4+dfsg-5.1) ...
Selecting previously unselected package dnsutils.
Preparing to unpack .../dnsutils_1%3a9.11.5.P4+dfsg-5.1_armhf.deb ...
Unpacking dnsutils (1:9.11.5.P4+dfsg-5.1) ...
Setting up libirs161:armhf (1:9.11.5.P4+dfsg-5.1) ...
Setting up dnsutils (1:9.11.5.P4+dfsg-5.1) ...
Processing triggers for libc-bin (2.28-10+rpil) ...
Processing triggers for man-db (2.8.5-2) ...
pi@raspberrypi:~ $
```

nslookup

Probably the most widely known utility for name/address referencing is “nslookup” you can call this command with either an IP address or a hostname on the command line, and it will query the DNS servers to get the complete information about that name or number. For example:

```
pi@raspberrypi:~ $ nslookup www.hva.nl
Server:          192.168.1.1
Address:         192.168.1.1#53

Non-authoritative answer:
www.hva.nl       canonical name = www.cms.hva.nl.
www.cms.hva.nl   canonical name = hvacms-prd-www.lb.uva.nl.
Name:   hvacms-prd-www.lb.uva.nl
Address: 145.18.12.47

pi@raspberrypi:~ $
```

In the output shown above, the first two lines returned after the command is entered give the name and address of the DNS server that the nslookup utility is asking for this information (yes, in this case the DNS server has a rather unusual name). The rest of the output gives you the complete DNS information for the host you asked about; here you can see that host actually has at least three different names.

You can also give an IP address to the nslookup utility, and it will return the name for that address:

```
pi@raspberrypi:~ $ nslookup 8.8.8.8
8.8.8.8.in-addr.arpa   name = dns.google.
```

This output shows that 8.8.8.8 is actually the IP address of a DNS server at Google!

If you give an invalid or unknown name on the command line, you will simply get a message that says the DNS server can't find any information about that name (NXDOMAIN is an abbreviation for “Non-Existant Domain”):

```
pi@raspberrypi:~ $ nslookup www.hva.ch
Server:          192.168.1.1
Address:         192.168.1.1#53

** server can't find www.hva.ch: NXDOMAIN

pi@raspberrypi:~ $
```

dig

The “dig” command is an alternative to “nslookup”. It works in much the same way, giving the name or IP address to be checked on the command line, but it gives back a lot more detailed information than nslookup does.

```
pi@raspberrypi:~ $ dig www.hva.nl

; <<>> DiG 9.11.5-P4-5.1-Raspbian <<>> www.hva.nl
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6008
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 4096
;; QUESTION SECTION:
;www.hva.nl.                IN      A

;; ANSWER SECTION:
www.hva.nl.                 28411   IN      CNAME   www.cms.hva.nl.
www.cms.hva.nl.            14451   IN      CNAME   hvacms-prd-www.lb.uva.nl.
hvacms-prd-www.lb.uva.nl.  3211    IN      A       145.18.12.47

;; Query time: 1 msec
;; SERVER: 192.168.1.1#53(192.168.1.1)
;; WHEN: Sat Sep 14 15:30:18 CEST 2019
;; MSG SIZE rcvd: 121

pi@raspberrypi:~ $
```

host

Another alternative to the “nslookup” and “dig” commands is the Linux “host” utility. This is used in much the same way as the other two, by calling it with a host name on the command line, but the information it returns is much simpler:

```
pi@raspberrypi:~ $ host www.hva.nl
www.hva.nl is an alias for www.cms.hva.nl.
www.cms.hva.nl is an alias for hvacms-prd-www.lb.uva.nl.
hvacms-prd-www.lb.uva.nl has address 145.18.12.47
pi@raspberrypi:~ $
```

ip

The “ip” utility provides a variety of information about the network configuration. The most common use of this command is to find out the IP address of the Linux system, using the “address” argument:

```
pi@raspberrypi:~ $ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group default qlen 1000
    link/ether dc:a6:32:03:7f:9a brd ff:ff:ff:ff:ff:ff
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether dc:a6:32:03:7f:9b brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.185/24 brd 192.168.1.255 scope global noprefixroute wlan0
        valid_lft forever preferred_lft forever
    inet6 2a02:120b:2c18:b380:a4c8:df67:a6c3:d8ef/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 266sec preferred_lft 26sec
    inet6 fe80::acad:a79c:77e6:24f8/64 scope link
        valid_lft forever preferred_lft forever
pi@raspberrypi:~ $
```

This output shows that this machine has three network interfaces; one for a “wired” ethernet cable connection (eth0), one for “wireless” or WiFi connection (wlan0), and one for a “loopback” connection (lo). For any of those interfaces which currently have an active connection, it also shows the “inet” (IPv4) information, and the “inet6” (IPv6) information.

The command option “address” can be shortened, it is often given as “ip ad”, but in fact can even be shortened to a single letter, “ip a”.

The “ip” utility is actually a relatively new replacement for several older utilities which were included in previous releases of Linux (and which were used in the FYS project in previous years). While the information they provide is approximately the same, the “ip” utility has been significantly updated, so it provides more relevant information and it works better with IPv6 connections. While you can currently still use “ifconfig”, we recommend using the new utility, because ifconfig will eventually not be included in the Linux distribution any more.

The “ip” command can also show the network routing configuration:

```
pi@raspberrypi:~ $ ip route
default via 192.168.1.1 dev wlan0 proto dhcp src 192.168.1.185 metric 303
192.168.1.0/24 dev wlan0 proto dhcp scope link src 192.168.1.185 metric 303
pi@raspberrypi:~ $
```

This shows your default gateway, and your subnet information. Again, the “route” option can be abbreviated, to a minimum of just “ip r”.

The ip command has many other options, which can display a lot of different kinds of information about the network on your machine. Check the man page for details, and try some things yourself!

Sockets

The “ss” command gives information about sockets. When run with no options, it prints a very large amount of information, so we usually add a pipe to “more” to present the information:

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
u_str	ESTAB	0	0	/run/systemd/journal/stdout	2038 * 0
u_str	ESTAB	0	0	/run/systemd/journal/stdout	9641 * 0
u_str	ESTAB	0	0	*	10180 * 0
u_str	ESTAB	0	0	*	10248 * 0
u_str	ESTAB	0	0	/run/systemd/journal/stdout	12075 * 0
u_str	ESTAB	0	0	/run/systemd/journal/stdout	12409 * 0
u_str	ESTAB	0	0	*	12466 * 0
u_str	ESTAB	0	0	*	12572 * 0
u_str	ESTAB	0	0	*	12683 * 0
u_str	ESTAB	0	0	*	12690 * 0
u_str	ESTAB	0	0	*	12714 * 0
u_str	ESTAB	0	0	/var/run/dbus/system_bus_socket	12738 * 0
u_str	ESTAB	0	0	*	12746 * 0
u_str	ESTAB	0	0	/var/run/dbus/system_bus_socket	12747 * 0
u_str	ESTAB	0	0	*	12808 * 0
u_str	ESTAB	0	0	/var/run/dbus/system_bus_socket	12816 * 0
u_str	ESTAB	0	0	/run/systemd/journal/stdout	12852 * 0
u_str	ESTAB	0	0	*	12870 * 0
u_str	ESTAB	0	0	*	12876 * 0
u_str	ESTAB	0	0	/run/user/1000/bus	12897 * 0
u_str	ESTAB	0	0	*	12920 * 0
u_str	ESTAB	0	0	*	12922 * 0

--More--

If you page through the output of this command, you will see information about all sorts of sockets on your machine. The problem here, of course, is understanding what they all are, and what all of the information given for each socket means.

We can use the Linux “grep” command to help with that understanding. Grep is basically a filtering program, so we can pipe the output of “ss” to the input of “grep”, and then give command line options which tell grep what information we want to see. For example, we could add “| grep LISTEN” to the ss command, so we get only a list of sockets which are listening for a connection:

u_str	LISTEN	0	0	/run/systemd/fck.progress	8058 * 0
u_str	LISTEN	0	0	/run/systemd/journal/stdout	8078 * 0
u_str	LISTEN	0	0	/var/run/dbus/system_bus_socket	12343 * 0
u_str	LISTEN	0	0	/run/avahi-daemon/socket	12347 * 0
u_str	LISTEN	0	0	/run/tdm.socket	12363 * 0
u_str	LISTEN	0	0	/tmp/ssh-Bh08HyxS352/agent.502	14216 * 0
u_str	LISTEN	0	0	/run/user/1000/systemd/private	14805 * 0
u_str	LISTEN	0	0	/run/user/1000/gnupg/S.gpg-agent.ssh	14811 * 0
u_str	LISTEN	0	0	/run/user/1000/gnupg/S.dirmgr	14814 * 0
u_str	LISTEN	0	0	/run/user/1000/gnupg/S.gpg-agent.browser	14816 * 0
u_str	LISTEN	0	0	/run/user/1000/gnupg/S.gpg-agent.extra	14818 * 0
u_str	LISTEN	0	0	/run/user/1000/bus	14820 * 0
u_str	LISTEN	0	0	/run/user/1000/gnupg/S.gpg-agent	14822 * 0
u_str	LISTEN	0	0	/run/user/1000/pmanfm-socket-0	14977 * 0
u_str	LISTEN	0	0	/run/user/1000/menu-cached-0	15055 * 0
u_str	LISTEN	0	0	/var/run/dhccpd.sock	15657 * 0
u_str	LISTEN	0	0	/var/run/dhccpd.unpriv.sock	15659 * 0
u_str	LISTEN	0	0	@tmp/.X11-unix/X0	16006 * 0
u_str	LISTEN	0	0	/tmp/.X11-unix/X0	16007 * 0
u_str	LISTEN	0	0	/tmp/ssh-KHNuBT9mStA/agent.578	16350 * 0
u_str	LISTEN	0	0	/tmp/OSL_PIPE_1000_SingleOfficeIPC_4917865a218918af6e8e08efe9c38a4	17895 * 0
u_str	LISTEN	0	0	/run/user/1000/.lxterminal-socket-0.0	18537 * 0
u_seq	LISTEN	0	0	/run/udev/control	8074 * 0

pi@raspberrypi:~\$

Another possibility would be to grep for “ESTAB”, which produces this output:

u_str	ESTAB	0	0	/run/systemd/journal/stdout	2038 * 0
u_str	ESTAB	0	0	/run/systemd/journal/stdout	9641 * 0
u_str	ESTAB	0	0	*	10180 * 0
u_str	ESTAB	0	0	*	10248 * 0
u_str	ESTAB	0	0	/run/systemd/journal/stdout	12075 * 0
u_str	ESTAB	0	0	/run/systemd/journal/stdout	12409 * 0
u_str	ESTAB	0	0	*	12466 * 0
u_str	ESTAB	0	0	*	12572 * 0
u_str	ESTAB	0	0	*	12683 * 0
u_str	ESTAB	0	0	*	12690 * 0

pi@raspberrypi:~\$

dhclient

Your machine will obtain an IP address as part of the Linux boot sequence, using DHCP (the Dynamic Host Configuration Protocol). You can use the “dhclient” utility to get some information about the DHCP configuration. This is again an administrative command, so it must be run using sudo.

When dhclient is run with no options it doesn’t produce any useful output:

```
pi@raspberrypi:~ $ sudo dhclient
RTNETLINK answers: File exists
pi@raspberrypi:~ $
```

However, if you add the “-v” option to the command line, things get interesting (check the man page for dhclient to see what the “-v” option means):

```
pi@raspberrypi:~ $ sudo dhclient -v
Internet Systems Consortium DHCP Client 4.4.1
Copyright 2004-2018 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/wlan0/dc:a6:32:03:7f:9b
Sending on   LPF/wlan0/dc:a6:32:03:7f:9b
Listening on LPF/eth0/dc:a6:32:03:7f:9a
Sending on   LPF/eth0/dc:a6:32:03:7f:9a
Sending on   Socket/fallback
DHCPREQUEST for 192.168.1.185 on wlan0 to 255.255.255.255 port 67
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 6
DHCPOFFER of 192.168.1.185 from 192.168.1.1
RTNETLINK answers: File exists
bound to 192.168.1.185 -- renewal in 36609 seconds.
pi@raspberrypi:~ $
```

Here you can see, for example, a DHCPACK (or DHCP Ack) message which shows that this machine got the IP address 192.168.1.185 from a DHCP server at 192.168.1.1, and that the IP address lease will expire in 36,609 seconds (a bit over 10 hours).

Linux User Accounts

There are two kinds of user accounts on Linux: "ordinary" users such as "pi" or "student", who have access only to their own files and directories and others which explicitly allow public access; and administrators (also known as "superusers") such as "root", who have access to all files and directories on the computer.

The Linux shell changes the command prompt to remind you which type of account you are currently using. As we have seen in this document until now, if you login as an ordinary user, the shell prompt will be a "\$", and if you login or "su" to root, the shell prompt will be "#":

```
pi@raspberrypi:~ $
```

```
root@raspberrypi:~#
```

The different shell prompt should remind you that you should be very careful when you are using the root account, because there are no file or directory restrictions on root. That means it is very easy to accidentally delete or overwrite files or directories, which can have serious negative consequences for your work, or even for your entire computer.

The general "rule of thumb" is that you should never login as root, because it is too dangerous. If you need to perform some commands which require root permissions, you should login to your ordinary user account, and then use either "su" or "sudo" to execute the root commands.

The difference between "su" and "sudo" is how long you will remain in the privileged (root) account; "su" actually starts a new shell with the root account, so you can work with it as long as you want, while "sudo" takes a specific command line argument and executes only that command as root, then returns you to your ordinary user account.

For security reasons, many Linux systems will not allow a user to simply "su" and get an unlimited root shell. On those systems, you have to use "sudo".

The difference between the ordinary "pi" account and the "root" account can be seen using the Linux "id" command, which prints the current account number and all groups that the account belongs to.

```
pi@raspberrypi:~ $ id
uid=1000(pi) gid=1000(pi) groups=1000(pi),4(adm),20(dialout),24(cdrom),27(sudo),
29(audio),44(video),46(plugdev),60(games),100(users),105(input),109(netdev),997(
gpio),998(i2c),999(spi)
pi@raspberrypi:~ $ sudo id
uid=0(root) gid=0(root) groups=0(root)
pi@raspberrypi:~ $
```

To illustrate the practical use of “sudo”, suppose that you need to edit the file /etc/host.conf. The permissions on that file normally do not allow an ordinary user to write it, so if you tried “nano /etc/host.conf”, you would get something like this:

```
GNU nano 4.4 /etc/host.conf
#
# /etc/host.conf - resolver configuration file
#
# Please read the manual page host.conf(5) for more information.
#
# The following option is only used by binaries linked against
# libc4 or libc5. This line should be in sync with the "hosts"
# option in /etc/nsswitch.conf.
#
order hosts, bind
#
# The following options are used by the resolver library:
#
multi on

[ File '/etc/host.conf' is unwritable ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^_ Replace   ^U Paste Text ^T To Spell  ^_ Go To Line
```

Here you can see that nano is telling you the file is unwritable, so even if you make changes in it you will not be able to save those changes to the original file. But if you use “sudo nano /etc/host.conf”, you would get something like this:

```
GNU nano 4.4 /etc/host.conf
#
# /etc/host.conf - resolver configuration file
#
# Please read the manual page host.conf(5) for more information.
#
# The following option is only used by binaries linked against
# libc4 or libc5. This line should be in sync with the "hosts"
# option in /etc/nsswitch.conf.
#
order hosts, bind
#
# The following options are used by the resolver library:
#
multi on

[ Read 16 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^_ Replace   ^U Paste Text ^T To Spell  ^_ Go To Line
```

The sudo command also has some significant advantages in configuration, control and logging compared to su. When you use su, you get a root shell prompt, so you can do pretty much anything you want, but when you use sudo you have to give the command to be executed on the command line, and the system administrator can configure what commands will be allowed. Also, the system will log whenever someone uses sudo, and what they used it for, so there is an audit trail. This can be an important part of system security. Because of this, some Linux distributions (such as Raspbian, Ubuntu, Linux Mint and others) do not allow ordinary users to use "su", so they have to use "sudo".

Linux File Permissions

Now that you know about Linux user accounts, and the differences between ordinary users and administrative users (superusers), you are ready to learn about File Permissions.

Permissions in Linux define three types of access:

- **Read:** Permission to read a file, or to list the contents of a directory
- **Write:** Permission to write (update or overwrite) a file, or create/delete files in a directory
 - Important note: The ability to *delete* a file is based on the permissions of the directory containing the file, not on the permissions of the file itself. That seems odd at first, but if you think about it, it makes sense. When you delete a file, you are actually updating the contents of the directory, not modifying the file itself.
- **Execute:** Permission to execute a file (run a program), or open the files in a directory
 - The concept of “execute” applied to a directory doesn’t actually make sense. In Linux, this has simply been used for a somewhat different purpose, to provide another layer of definition in accessing directories and the files contained in them.

These three types of permission are then defined for three different categories of users:

- **Owner:** The user who owns the file
- **Group:** Users who are in the same group as the file
- **Others:** Users who are not in either of the previous two categories

The file permissions then consist of the three categories of users, with the three types of access defined for each category of users. When you look at a “long” directory listing (`ls -l`) for a file, you might see:

```
pi@raspberrypi:~ $ ls -l procs.txt
-rw-r--r-- 1 pi pi 7517 Sep 14 16:30 procs.txt
pi@raspberrypi:~ $
```

The first character on the output line (“-”) has a special meaning which we will ignore for now.

The next three characters define the read, write and execute permissions for the owner of the file, with the obvious abbreviations for granted permissions, and “-” for lack of permission.

The next three characters show the same information for users in the group of the file.

The next three characters show the same information for all other users on the system.

So the file above has read/write permission for the owner, and read-only permission for those in the same group as the file, and also for anyone else.

Managing Operating System Services

The Linux operating system includes a large number of “services”, which are processes that are started automatically on system boot to provide various types of functionality to the users. Ordinary users often take such services for granted, but Linux administrators have to be aware of them, and know how to take certain actions such as starting (or re-starting) and stopping individual services.

The “service” command is used to manage system services, but it is an administrative command, so it must be run via the “sudo” utility – it will not work if it is run from an ordinary user account. To get a list of the currently defined system services, you can use the “--status-all” option. However, there are likely to be more services defined than can be shown in a single screen, so you will want to pipe the output of this command to “more”:

```
[ - ] apparmor
[ + ] avahi-daemon
[ + ] bluetooth
[ - ] console-setup.sh
[ + ] cron
[ + ] dbus
[ + ] dhcpcd
[ + ] dphys-swapfile
[ + ] fake-hwclock
[ - ] hwclock.sh
[ - ] keyboard-setup.sh
[ + ] kmod
[ + ] lightdm
[ + ] networking
[ - ] nfs-common
[ - ] paxctld
[ - ] plymouth
[ - ] plymouth-log
[ + ] procps
[ + ] raspi-config
[ ? ] rng-tools
[ - ] rpcbind
[ - ] rsync
--More--
```

The status code shown by the service command is “+” for running services, “-” for stopped services, and “?” for services which do not support a status query, so it is not possible to determine the status.

The length of this list should give you a good idea of how many services are running on your machine, and how important it is to be able to manage them properly.

One of the system services that you might want to control is the Apache web server, which is known in the service list as “apache2”. The most common action that you are likely to need to take is to restart the service, for example when you have modified one of the Apache configuration files.

```
sudo service apache2 restart
```

You can also check the status of the web server:

```
sudo service apache2 status
```

There are other “service” command options, for example to start or stop a service; you should refer to the man page for more details about them.